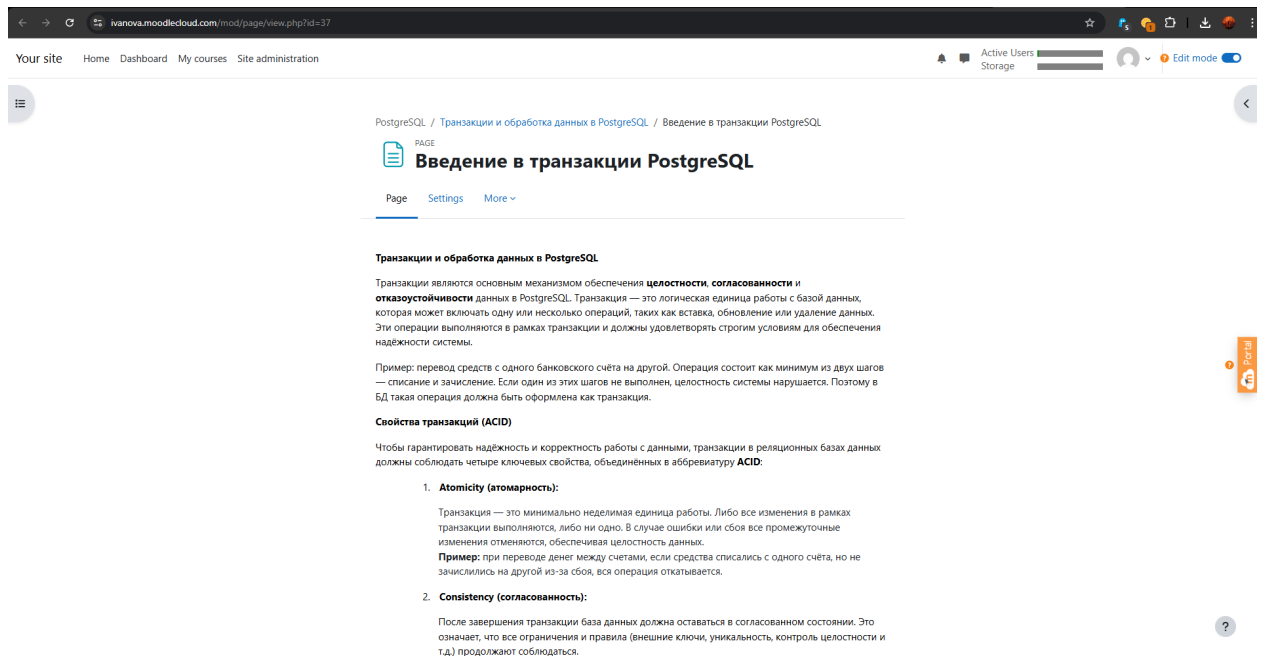
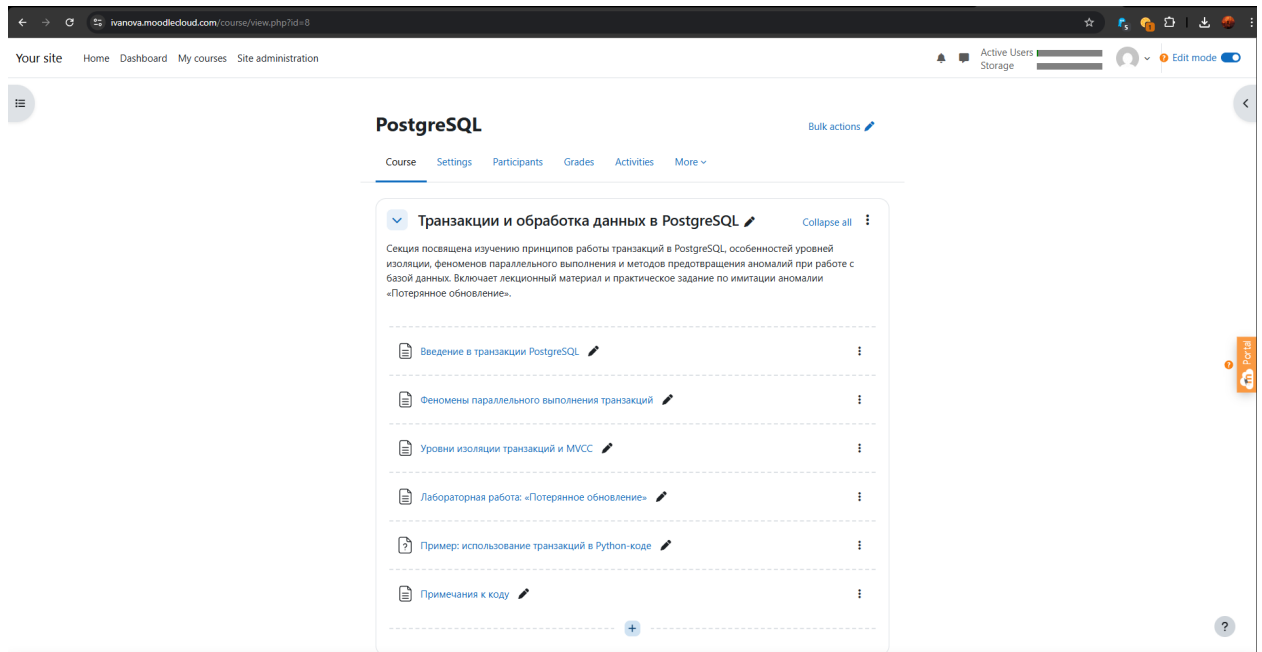


Задание 3.3. Создайте фрагмент электронного курса (1 занятие) с использованием, например сервиса moodle.

Teachbase запрашивает личные данные и принудительную регистрацию с подключением через менеджера, поэтому было принято решение разработать фрагмент электроного курса на основе Moodle.



ivanova.moodlecloud.com/mod/page/view.php?id=38

Your siteHomeDashboardMy coursesSite administration

Active UsersStorage

Edit mode

PostgreSQL / Транзакции и обработка данных в PostgreSQL / Феномены параллельного выполнения транзакций

PAGE

Феномены параллельного выполнения транзакций

PageSettingsMore

Феномены параллельного выполнения транзакций

Хотя ACID требует изоляции, на практике системы управления базами данных реализуют её с разной степенью строгости. Это приводит к различным аномалиям при параллельном выполнении транзакций. Эти аномалии объясняют, зачем нужны уровни изоляции, описанные в SQL-стандарте, и какие последствия возможны при их нарушении:

1. Потерянное обновление (Lost Update)

Если разные транзакции меняют одни и те же данные, то может возникнуть ситуация при которой изменения одной транзакции перезапишутся другой транзакцией. Например есть две транзакции. Они производят обновление одной и той же строки. Вторая транзакция изменила строку и зафиксировала свои изменения. После этого зафиксировала свои изменения первая транзакция и перезаписала изменения второй транзакции.

Пример:



ivanova.moodlecloud.com/mod/page/view.php?id=39

Your siteHomeDashboardMy coursesSite administration

Active UsersStorage

Edit mode

PostgreSQL / Транзакции и обработка данных в PostgreSQL / Уровни изоляции транзакций и MVCC

PAGE

Уровни изоляции транзакций и MVCC

PageSettingsMore

3. Уровни изоляции

Чтобы контролировать допустимые взаимодействия параллельных транзакций и управлять феноменами, описанными выше, SQL и PostgreSQL определяют **уровни изоляции транзакций**. Каждый уровень задаёт допустимую степень «видимости» изменений, внесённых другими транзакциями:

1. **READ UNCOMMITTED**: согласно стандарту SQL, на этом уровне допускается грязное чтение — т.е. чтение данных, изменённых, но ещё не зафиксированных другими транзакциями. Однако в PostgreSQL грязное чтение невозможно, даже на этом уровне. Поэтому READ UNCOMMITTED фактически работает идентично уровню READ COMMITTED. Это означает, что данные, изменённые, но не зафиксированные другими транзакциями, никогда не будут видны.

2. **READ COMMITTED** (по умолчанию в PostgreSQL): при этом уровне каждая SQL-команда в пределах транзакции видит только те изменения, которые были зафиксированы до начала выполнения этой команды. Это означает, что разные команды внутри одной транзакции могут видеть разное состояние данных. Возможны такие феномены, как **неповторяющееся чтение** и **фантомное чтение**, но **грязное чтение исключено**. Благодаря MVCC в PostgreSQL чтение не блокирует запись, и наоборот — что делает этот уровень достаточно производительным при высокой конкуренции.

3. **REPEATABLE READ**: Все запросы внутри транзакции видят **одинаковое состояние данных**, зафиксированное в момент первого запроса. Это устраняет **неповторяющееся чтение** и **грязное чтение**. В PostgreSQL этот уровень также предотвращает **фантомные чтения**, что достигается за счёт **снимка базы данных (snapshot isolation)**. Транзакция не видит изменений, сделанных другими транзакциями после создания снимка. Если транзакция попытается изменить строки, которые были изменены и зафиксированы другой транзакцией после момента создания снимка, произойдёт конфликт записи, и операция завершится

ivanova.moodlecloud.com/mod/page/view.php?id=40

Your site Home Dashboard My courses Site administration

Active Users Storage Edit mode

PostgreSQL / Транзакции и обработка данных в PostgreSQL / Лабораторная работа: «Потерянное обновление»

PAGE

Лабораторная работа: «Потерянное обновление»

Page Settings More

Методические рекомендации по лабораторной работе

Тема: Имитация аномалии «Потерянное обновление» на примере управления запасами книг в библиотеке

Цель работы

Демонстрировать, как при недостаточно строгом уровне изоляции транзакций в базе данных возможно возникновение аномалии «потерянное обновление». Показать на практике, как параллельные операции могут влиять на корректность данных, и какие методы существуют для предотвращения этой аномалии.

Предметная область

Рассматривается система учёта количества книг в филиалах библиотеки. Каждая книга может находиться в одном или нескольких филиалах одновременно, и в каждом из них хранится определённое количество экземпляров. В системе реализована возможность перемещения книг из одного филиала в другой — такая операция уменьшает количество книг в филиале-отправителе и увеличивает их в филиале-получателе.

В лабораторной работе студенты моделируют ситуацию, при которой два параллельных процесса одновременно пытаются переместить книги из одного и того же филиала. Это может привести к ошибочному уменьшению количества книг, если обновления «перетрут» друг друга — типичная ситуация, иллюстрирующая аномалию потерянного обновления.

Задачи лабораторной работы

1. Проектирование и создание структуры базы данных

В начале работы необходимо создать следующие сущности:

ivanova.moodlecloud.com/mod/page/view.php?id=42

Your site Home Dashboard My courses Site administration

Active Users Storage Edit mode

PostgreSQL / Транзакции и обработка данных в PostgreSQL / Примечания к коду

PAGE

Примечания к коду

Page Settings More

Управление транзакциями: механизмы SQL и практика в PostgreSQL

Практическая часть: управление транзакциями в PostgreSQL на примере Python и psycopg2

В PostgreSQL, как и в других реляционных СУБД, транзакции управляются через три базовые SQL-команды:

- **BEGIN** (или **START TRANSACTION**) — открывает новую транзакцию. Все изменения, сделанные после этой команды, считаются частью одной логической операции.
- **COMMIT** — завершает транзакцию, делая изменения постоянными в базе данных.
- **ROLLBACK** — отменяет все изменения, произведённые после BEGIN, в случае ошибки или отмены операции.

Эти команды особенно важны в сценариях, где ошибки могут возникнуть в произвольный момент выполнения — например, при валидации данных, сетевых сбоях или нарушении ограничений (например, уникальности или CHECK).

Транзакции позволяют гарантировать, что либо все действия выполняются успешно, либо не сохраняется ничего, тем самым обеспечивая атомарность и согласованность (ACID).

Пример: использование транзакций в Python-коде

Для демонстрации мы используем библиотеку `psycopg2`, которая предоставляет интерфейс к PostgreSQL из Python. В рамках практической части нас интересует только функция `seed_accounts_transactional`, так как именно она демонстрирует транзакционную логику. Функции `clear_db()` и `print_table()` необходимы лишь для удобства тестирования и вывода результата, но к теме транзакций непосредственно не относятся.

Ссылка на ресурс: <https://ivanova.moodlecloud.com/course/view.php?id=8>